Clean Code

What is clean code?

Clean code is code that is simple, easy to read, and easy to understand by any developer, not just the person who wrote it. It is not just about making the code work; it is about making it neat and well-organized.

Why does it exist?

Clean Code exists to solve many software development problems such as:

- Hard-to-read code
- Difficult debugging
- Slow development
- Confusing project structure
- High maintenance cost

When code is clean:

- Teams work faster
- Bugs become easier to fix
- Projects become easier to expand and maintain

Examples where useful:

- Not clean code:

```
int x(int a,int b){
return a+b;
}
```

- Clean code:

```
int calculateSum(int firstNumber, int secondNumber) {
   return firstNumber + secondNumber;
}
```

Project Structure

What is it?

Project structure is how you organize your files and folders. The organization should match the idea and purpose of the project, making it easy to navigate.

Examples:

- **Structure by Type (Layer-based)**:

Main folders:

- models/: For data blueprints (Data Models).
- views/ or screens/: For UI designs and screen.
- controllers/ or blocs/: For logic and state management.

When to use it: Good for very small or simple projects.

- **Structure by Feature (Feature-based)**

Main folders:

- features/auth/: Contains everything for Login, Signup, and Forgot Password.
- features/home/: Contains the home screen and its specific widgets.
- features/profile/: Contains the user profile screen and its settings.

When to use it: Great for medium to large projects because it makes teamwork easier; developers can work on separate features without breaking each other's code.

- **Layered Clean Architecture**

Main folders:

- data/ layer: Handles getting data from the internet (APIs) or local storage (like SQLite).
- domain/ layer: The core "brain" and business rules of the app. It is completely independent of the UI.
- presentation/ layer: Handles the UI widgets and state management (like BLoC or Provider).

When to use it: Used in production-grade apps that need heavy testing (Unit Tests) and long-term scaling.

Namings

Naming means choosing clear and meaningful names for your variables, functions, files, and folders. The name should tell you exactly what that item is or what it does.

- Good naming helps developers:
  - Understand code quickly
  - Reduce mistakes
  - Improve readability
- Examples:
  - Variables:
    - Bad: let d = 30; (What is 30?)
    - Good: let daysSinceLastLogin = 30;
  - Functions:
    - Bad: function check(u) { ... }
    - Good: function isUserActive(user) { ... }

DRY — Don't Repeat Yourself:

DRY is a basic rule in programming that means you should not copy and paste the same code in multiple place. Every piece of logic should only be written once.

- DRY helps by:
  - Reducing repetition
  - Making updates easier
  - Improving maintainability

- Examples:
  - **Validation Functions**: Instead of writing the email validation logic inside both the "Login" page and the Sign Up" page, you write it once in a shared function" and reuse it.

- **UI Component**s: In modern frameworks like React, you create one customizable Button component and use it everywhere on the website, instead of rewriting HTML and CSS for every single button.